TEMA: Sistema de plantilla en Django

Resultados de Aprendizaje:

- Que los estudiantes comprenden los diferentes sistemas de etiquetas para plantillas que posee Django para la generación de aplicaciones

DESARROLLO

Djanog posee una serie de etiquetas que podemos utilizar y filtos que podemos utilizar dentro de los archivos html

1. **if/else**
   La etiqueta {% if %} evalúa una variable, y si esta es ‘‘true’’ el sistema mostrará todo lo que hay entre {% if%} y {% endif %}, esta tambine acepta **and or , not** por ejemplo:

```
{% if es_fin_de_semana %}
   <p>¡Bienvenido fin de semana!</p>
{% else %}
   <p>De vuelta al trabajo.</p>
{% endif %}
{% if lista_atletas and lista_entrenadores %}
   Atletas y Entrenadores están disponibles
{% endif %}
```

   En Python y en el sistema de plantillas de Django, los siguientes objetos son evaluados como False (falsos) en un contexto booleano:
   - Una lista vacía ([])
   - Una tupla vacía (())
   - Un diccionario vacío ({})
   - Una cadena vacía ('')
   - El cero (0)
2. **For**
   La etiqueta {% for %} permite iterar sobre cada uno de los elementos de una secuencia

```
<ul>
{% for atleta in lista_atletas %}
   <li>{{ atleta.nombre }}</li>
{% endfor %}
</ul>
```

3. **Filtros**
   Los filtros de plantillas son formas simples de alterar el valor de una variable antes de mostrarla.
   a. **{{ nonmbre|lower }}**
      Esto muestra el valor de {{ nombre }} después de aplicarle el filtro lower, el cual onvierte el texto a minúscula.
4. **Encadenamiento de filtos**
   a. Aquí un ejemplo que toma el primer elemento de una lista y la convierte a mayusculas:
      {{ mi_lista|first|upper }}
   b. **{{ bio|truncatewords:"30" }}**
      Esto muestra las primeras 30 palabras de la variable bio.
5. **Fomateo de campo de tipo fecha**
   a. {{ fecha_ingreso|date:"d/m/Y" }}
6. **Inclusion**
   Inclulle el contenido de una plantilla dentro de otra
   {% include "menu.html" %}

Herencia de plantillas

Cuando estamos trabajando con django en ocasiones necesitamos crear una pantilla en particuar y que las demas hereden este dieño de esta manera no ahorraremos mucho trabajo

{% extends "base.html" %}

Esto indica que que pantilla estamos heredando

Etiqueda de bloque

Cuando realizamos la plantilla base deberemos definir los bloquede de codigo conde se insertara la informacion para esto utilizamos la etiqueta

{% block contenido %}{% endblock %}

Ejemplo de herencia de plantillas

```
<html lang="en">
<head>
    <title>{% block title %}{% endblock %}</title>
</head>
<body>
    <h1>Mi sitio Web</h1>
    {% block content %}{% endblock %}
    {% block footer %}
    <hr>
    <p>Gracias por visitar nuestro sitio web.</p>
{% endblock %}
</body>
</html>
```

Utilizacion de la herencia

```
{% extends "base.html" %}

{% block title %}La fecha actual{% endblock %}

{% block content %}
   <p>Hoy es: {{ fecha_actual }}</p>
{% endblock %}
```

Implementacion de plantillas para el proyecto comercial

Requerimientos:

Para la realizacion de esta guia usted debe de contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guia anterios
- Contar con un servidor de base de datos mysql

Plantilla de dijango

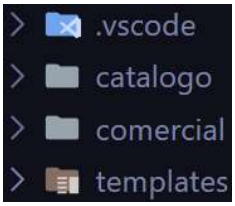
1. Lo primero que aremos es modificar el archivo **catalogo/view/categoria.**py , que hemos creado en anteriores guias , para enviar los datos que estan almanenados en la basse de datos
2. Quedando nuestro archivo de la sigueinte manera , esot nos permitirea enviar todos los datos de categori almacenados en la base de datos

```python
from django.shortcuts import render
from ..models.categoria import Categoria

# Create your views here


def lista_categorias(request):
    lista_categorias = Categoria.objects.all()
    return render(request, "categoria/lista.html", context={
        'categorias': lista_categorias
    })
```

3. Hoy crearemos nuestra plantilla base que embolvera a los demas tempalte , como esta plantilla se podra utilizar en todas las app que usted puede crear , por ello la pondremos fuera de la app categoira , la pondremos a la par de las carpetas catalogo y comercial , y crearemos una carpetla llamada templates



4. Creamos el archivo base **teemplates/base.html**



5. Agregamos nuetros codigo html en el arcivo base.html

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Comercial - {% block titulo %}{% endblock %}</title>
    <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  </head>

  <body class="min-h-screen bg-gray-200 text-slate-800">
    <div class="mx-auto w-full max-w-7xl px-4">
      <header >
        <nav class="flex items-center gap-x-8 py-2 ">
          <a
            href="{% url 'list_categoria' %}"
            class="text-2xl font-bold text-blue-600"
          >
            Comercial
          </a>

          <div class="flex items-center gap-6 ">
            <a
              href="{% url 'list_categoria' %}"
              class="text-base font-medium text-gray-500 transition-colors hover:text-blue-600 sm:text-lg"
            >
              Categoría
            </a>
          </div>
          <a

            class="ml-auto text-base font-medium text-gray-500 transition-colors hover:text-blue-600 sm:text-lg
            cursor-pointer"
          >
            Cerrar sesión
          </a>
        </nav>
      </header>

      <main class="w-full  py-2 sm:py-4">
        {% block contenido %} {% endblock contenido %}
      </main>
    </div>
  </body>
</html>
```

Es imporante destcar los boques {% block%} que nos permira agergar inforamcion desde otra plantilla

6. Por defecto django busca los template en la carpeta tempate de cada app , pero como nostros hemos creado fura de al app tendremos que indicarle que busque en el directorio que hemos creado para ello debemos de modifica el archivo **comercial/settings.py**
7. Debera de importar la librería del sistemas os , al comienzo del archivo

```
from pathlib import Path
import os
```

8. Deberemos agregar el directorio que hemos creado en la variable templates , si ha puesto otro nombre debera de poner dicho nombre

```python
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
            os.path.join(BASE_DIR, "templates")
        ],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

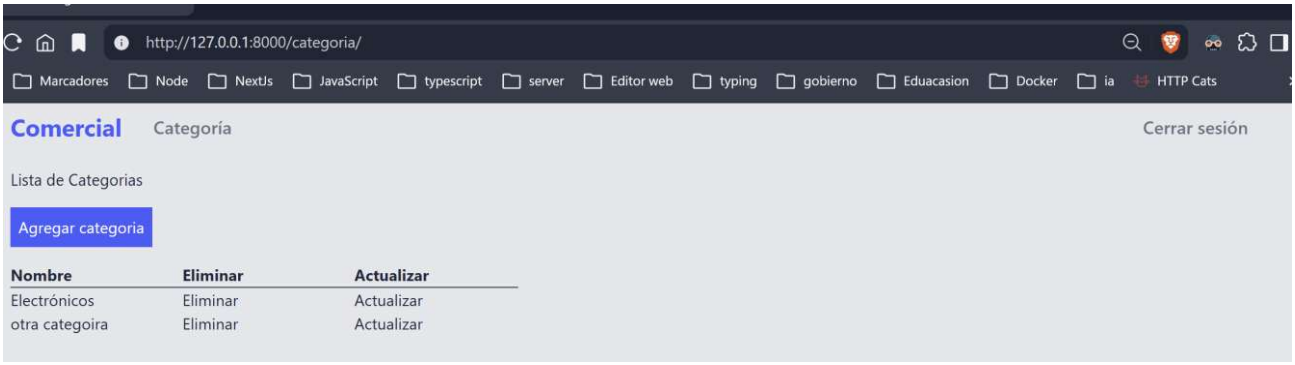
9. Para finalalizar editaremos la pantilla **catalogo/templates/lista.html ,** de la siguiente manera

```html
{% extends "base.html" %}
{% block titulo %} Categoria {% endblock titulo %}

{% block contenido %}
<h1 class="font-bold text-balance">Lista de Categorias</h1>
<a href="#" class="p-2 bg-blue-600 my-4 inline-block text-white rounded hover:bg-blue-700">Agregar categoria</a>
<div class="grid grid-cols-3 gap-2 border-b max-w-lg font-bold">
  <p>Nombre</p>
  <p>Eliminar</p>
  <p>Actualizar</p>
</div>
{% if categorias %}
{% for categoria in categorias %}
<div class="grid grid-cols-3 gap-2 max-w-lg hover:bg-gray-300 ">
  <p>{{categoria.nombre}}</p>
  <a href="#" class="cursor-pointer hover:text-blue-600 ">Eliminar</a>
  <a href="#" class="cursor-pointer hover:text-blue-600 ">Actualizar</a>
</div>
{% endfor %}
{% else %}
  <p>No hay categorias registradas </p>
{% endif %}

{% endblock contenido %}
```

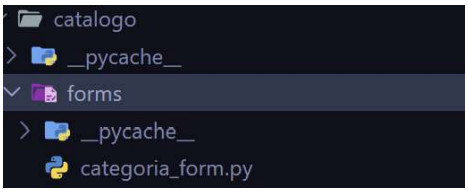
10. Si corremos la aplicasion en la url de categoiras podemos observer nuesto diseño



Creacion de formularios

Para teminar de realizar las operaciond del crud de la categoria debemos de crear nuestros formulario muy similar a los modes utilizados para definir la base de datos

11. Creamos una los archivos **catalogo/forms/catagoria_form.py** , como se muestra en la imagen



12. Editamos el archivo **catalogo/forms/catagoria_form.py**

```python
from django import forms
from django.utils import timezone
from ..models.categoria import Categoria


class CategoriaForm(forms.Form):
    nombre = forms.CharField(max_length=255, label="Nombre", required=True)

    descripcion = forms.CharField(
        max_length=255,
        label="Descripción",
        required=False,
        widget=forms.Textarea(attrs={
            "rows": 3
        }))

    def save(self):
        Categoria.objects.create(
            nombre=self.cleaned_data['nombre'],
            descripcion=self.cleaned_data['descripcion'],
            created_at=timezone.now(),
            updated_at=timezone.now()
        )

    def update(self, categoria):
        categoria.nombre = self.cleaned_data['nombre']
        categoria.descripcion = self.cleaned_data['descripcion']
        categoria.updated_at = timezone.now()
        categoria.save()
```

Explicasion del codigo anteriro

- Define los elemtnos del formulario , tiene que considerar los valor definidos en el modelo , la porpiedad **label** indica el texto que se mostrar el el formulario , **widget** indica que elemento correspondera al html , muy importa la funcon **attrs** indica los atributos del htlm , en este apartado puedo incluir **class** para definir sus proprias calses o el **placeholder** , para indicar un texto por defecto

```python
nombre = forms.CharField(max_length=255, label="Nombre", required=True)

descripcion = forms.CharField(
    max_length=255,
    label="Descripción",
    required=False,
    widget=forms.Textarea(attrs={
        "rows": 3
    }))
```

- Atrabes de este formularios podemos guardar directamente en la base de datos , con la funcon save, la funcion .**cleaned_data** permite capturar los dartos enviado por el formulario

```python
def save(self):
    Categoria.objects.create(
        nombre=self.cleaned_data['nombre'],
        descripcion=self.cleaned_data['descripcion'],
        created_at=timezone.now(),
        updated_at=timezone.now()
    )
```

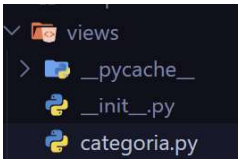
- La funcon update nos permite actuaziar una categoria , tenga en cunta que resibe como parameto la categoria que desea actualizar

```python
    )

def update(self, categoria):
    categoria.nombre = self.cleaned_data['nombre']
    categoria.descripcion = self.cleaned_data['descripcion']
    categoria.updated_at = timezone.now()
    categoria.save()
```

Generacion de vistas

13. Para poder realziar nuestro CRUD de categorias deberemos de modificar le archivo **catalogo/views/categoria.py** y agregare todos las funciones necearias para poder agergar , eliminar y actualziar una categoira



14. Funcionavilidades de crud de categoria

```python
from django.shortcuts import render, redirect, get_object_or_404
from ..models.categoria import Categoria
from ..forms.categoria_form import CategoriaForm
# Create your views here


def lista_categorias(request):
    lista_categorias = Categoria.objects.all()
    return render(request, "categoria/lista.html", context={
        'categorias': lista_categorias
    })


def delete_categoria(request, id):
    categoria = get_object_or_404(Categoria, id=id)
    categoria.delete()
    return redirect('list_categoria')
```

```python
def add_categoria(request):
    if request.method == 'POST':
        form = CategoriaForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('list_categoria')
        else:
            return render(request, "categoria/add.html", context={
                'form': form
            })
    return render(request, "categoria/add.html", context={
        'form': CategoriaForm()
    })
```

```python
def update_categoria(request, id):
    categoria = get_object_or_404(Categoria, id=id)
    if request.method == 'POST':
        form = CategoriaForm(request.POST)
        if form.is_valid():
            form.update(categoria)
            return redirect('list_categoria')
        else:
            return render(request, "categoria/update.html", context={
                'form': form
            })

    form = CategoriaForm(initial={
        'nombre': categoria.nombre,
        'descripcion': categoria.descripcion
    })
    return render(request, "categoria/update.html", context={
        'form': form,
        'categoria': categoria
    })
```

Explicasion de las funciones

- **Eliminar categoira**

  ```python
  def delete_categoria(request, id):
      categoria = get_object_or_404(Categoria, id=id)
      categoria.delete()
      return redirect('list_categoria')
  ```

  Esta funcion nos permite eliminar una categoria dado el id pero lo mas testacado es el metodo **get_object_or_404(Categoria, id=id)** que nos permite extraer un elemeto de la base de datos y no exitir retorna un 404 no encontrado de forma automatica
  El otro puto a destcar es **redirect** que nos permite redirigir a una ur nombrada esta esta en el archivo url.py , en lo sigueinte pasos mirar algo simular por el momento aun no existiran algunas url , pero las crearemos en pasos siguienetes

- **Vista para guardar**

```python
def add_categoria(request):
    if request.method == 'POST':
        form = CategoriaForm(request.POST)
        # valíamos que el se cumples todas la condiciones del formulario
        if form.is_valid():
            # guarda el la base de datos
            form.save()
            return redirect('list_categoria')
        else:
            # si no esta valido lo retorna al mismos lugar
            return render(request, "categoria/add.html", context={
                'form': form
            })
    # si no es el meto post , creamos un formulario limpio y lo enviamos
    return render(request, "categoria/add.html", context={
        'form': CategoriaForm()
    })
```

Esta funcion cumple dos putos inportantes , verifica sie el motodo es POST , para guardar el formulario en la base de datos , de lo contrario crea un formualrio el blacon que es enviado al template , denote que todo formulario de djanog tiene el metodo is_valid , que sirve para verificar que los campos del formulario sean validos , y si no lo esta el formulario de manera interna posee los error de cada campo para ser enviado al template , **form = CategoriaForm(request.POST) ,** extraer y crear un formulario con los datos que viene por el post

- **Actualizar**

```python
def update_categoria(request, id):
    categoria = get_object_or_404(Categoria, id=id)
    if request.method == 'POST':
        form = CategoriaForm(request.POST)
        if form.is_valid():
            form.update(categoria)
            return redirect('list_categoria')
        else:
            return render(request, "categoria/update.html", context={
                'form': form
            })

    form = CategoriaForm(initial={
        'nombre': categoria.nombre,
        'descripcion': categoria.descripcion
    })
    return render(request, "categoria/update.html", context={
        'form': form,
        'categoria': categoria
    })
```

Se revisa si no es el metod post , buscamos la categoira e inicializamos el fomrulario con datos pertienentes y es enviado al temple correpondiente dichos temple aun no existem pero los realizaremos en pasos sigueintes

Generacion de url para el CRUD de categoria

15. Definimos las url de categlorias modificando el archivo **catalogo/urls.py**

```python
from django.urls import path
from .views.categoria import lista_categorias, delete_categoria, add_categoria, update_categoria

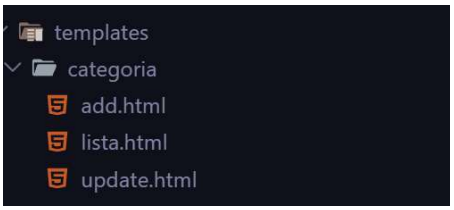


urlpatterns = [
    # Categorias
    path("categoria/", lista_categorias, name="list_categoria"),
    path("categoria/agregar/", add_categoria, name="add_categoria"),
    path("categoria/actualizar/<int:id>/",
         update_categoria, name="update_categoria"),
    path("categoria/eliminar/<int:id>/",
         delete_categoria, name="delete_categoria"),
```

<int:id> indica que el tipo de datos a enviara por el path sera de tipo entero

Creacion de template

16. Para nuestro CRUD de categoria deberemos de crear los template , de las cuales ya poseemos el archivo lista



17. Modificamos el arhivo **catalogos/templates/categira/lista.html** , para poder enlazar por medio de las url la posivilidad de poder elininar , agregar y actualizar , tome en cuenta que la fucno **"{% url "add_categoria" %}"** permite hacer referencia a una url nombrada en el archivo urls.py

```
{% extends "base.html" %}
{% block titulo %} Categoria {% endblock titulo %}

{% block contenido %}
<h1 class="font-bold text-balance">Lista de Categorias</h1>
<a href="{% url "add_categoria" %}" class="p-2 bg-blue-600 my-4 inline-block text-white rounded
hover:bg-blue-700 ">Agregar Categoria</a>
<div class="grid grid-cols-3 gap-2 border-b max-w-lg font-bold">
  <p>Nombre</p>
  <p>Eliminar</p>
  <p>Actualizar</p>
</div>
{% if categorias %}
{% for categoria in categorias %}
<div class="grid grid-cols-3 gap-2 max-w-lg hover:bg-gray-300 ">
  <p>{{categoria.nombre}}</p>
  <a href="{% url 'delete_categoria' categoria.id %}" class="cursor-pointer hover:text-blue-600 ">Eliminar</a>
  <a href="{% url 'update_categoria' categoria.id %}" class="cursor-pointer hover:text-blue-600 ">Actualizar</a>
</div>
{% endfor %}
{% else %}
  <p>No hay categorias registradas </p>
{% endif %}

{% endblock contenido %}
```

18. Modificamos el archivo **catalogos/templates/categira/add.html ,** que nos permitira agergar una categoria , tenga en cuenta que para eto utilizaremos el formulario creado con anterioridad , pero

para poder renderizar el formulario utilizaremos la funcion **{{form.as_p}}** que renderiza el formulario pero encerrado entre etiquetas ,

```
<p>
  <label for="id_nombre">Nombre:</label>
  <input type="text" name="nombre" maxlength="255"
  required id="id_nombre">
</p>
```

tambien podemos utilizar **{{form.as_ul}}** esto lo que hara es renderizar cada elemento del formulario en una etiqueta

```
<li> == $0
  ::marker
  <label for="id_nombre">Nombre:</label>
  <input type="text" name="nombre" maxlength="255"
  required id="id_nombre">
</li>
```

19. Modifiquemos el archivo **catalogos/templates/categira/add.html**

```
{% extends "base.html" %}

{% block titulo  %} agregar Categoria{% endblock titulo %}
{% block contenido %}
    <h1>Agregar Categorias</h1>
    <form method="post" action="{% url 'add_categoria' %}">
      {% csrf_token %}
      {{form.as_p}}
      <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700 rounded">Guardar</button>
    </form>
{% endblock contenido %}
```

Es importante destacar que cada vez que genermos un formulario tengremos que agregar la etiqueta {% csrf_token %} esta nos permitira evitar alguno tipos de ataques a nuetra web

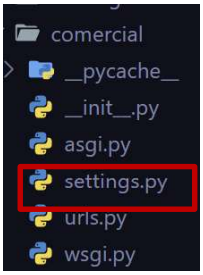
20. Hoy modificaremos el archivo **calogos/templates/categira/update.html ,** si ustes se percata es muy similar al archivo de add.html , con la unica diferencia es dondo redirige la informacion

```
{% block titulo  %} agregar Categoria{% endblock titulo %}
{% block contenido %}
    <h1>Actualizar Categoria</h1>
    <form method="post" action="{% url 'update_categoria' categoria.id %}">
      {% csrf_token %}
      {{form.as_p}}
      <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700
      rounded">Guardar</button>
    </form>
{% endblock contenido %}
```

21. Ya puede probar la aplicasion , pero si es observador e inteta agerga inforamcion no valida al formulario este le mostrar alguno errores de manera automatica , con la diferencia que estan en ingles

Cambiando mensajes a español

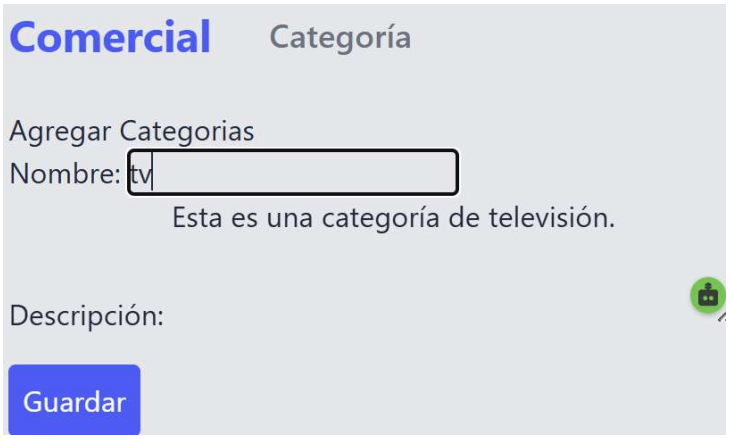
22. **Django** soporta una gran variedad de lenguajes y entre ellos esta el español, para poder cambiar el lenguaje de los menajes deberá de modifica el archivo **comercial/settings.py**



23. Busca la exprecion LANGUAGE_CODE y la deja en español

```
LANGUAGE_CODE = 'es'
```

24. Aun que ya nuesta aplicasion eta funcional tenemos un pequeño problem y es que los formularios no son tan agredables de ver



25. Para solucionar esto utilizaremos una librería que nos ayudara a añadir diseños utilizando tailwind , esta es **Crispy Tailwind**
26. para instalar esta librería utilizaremos el comando **pip install crispy-tailwind,** si usted prefiere utilizar bootstrap tambien exites una librería para ello llamda crispy-bootstrap5
27. una vez instalada la librería deberemos de configurar nuestro proyecto para que la soporte , para ello debemos de modifica el archivo **comercial/settings.py ,** donde definimos el lenguaje y agregar lo siguiente , siempre antes que nuestras aplicasiones

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    "crispy_forms",
    "crispy_tailwind",
    'catalogo'
]
```

28. ademas deberemos de agergar lo sigueinte al final del archivo
CRISPY_ALLOWED_TEMPLATE_PACKS = "tailwind"

CRISPY_TEMPLATE_PACK = "tailwind"

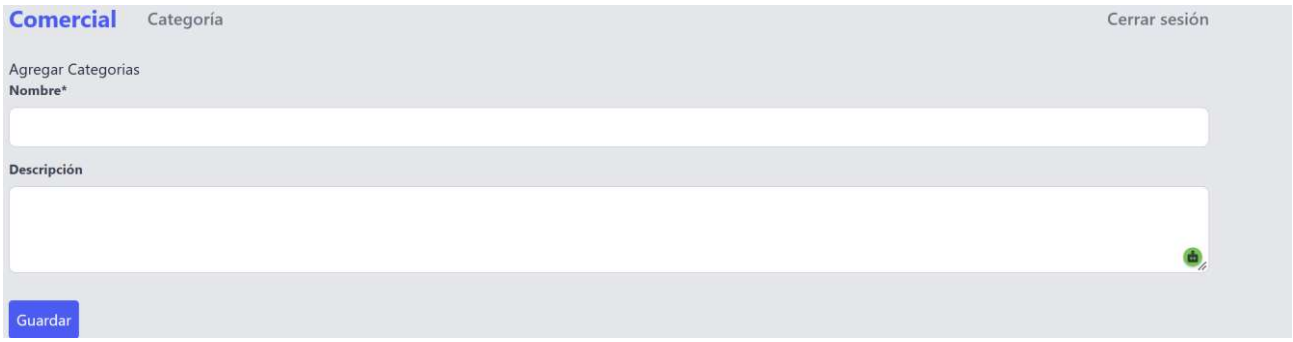
29. con lo anteriro configurado tendremos que hacer unos pequeños cambios a los templete add.html y update.html

modificando los template

30. Modificamos el template add.html

```
{% extends "base.html" %}
{% load tailwind_filters %}
{% block titulo %} agregar Categoria{% endblock titulo %}
{% block contenido %}
    <h1>Agregar Categorias</h1>
    <form method="post" action="{% url 'add_categoria' %}">
        {% csrf_token %}
        {{form|crispy}}
        <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700 rounded">Guardar</button>
    </form>
{% endblock contenido %}
```

31. Con esos pequeños cambios obtendremos un resultado como el siguiente



32. hoy modificaremos el templete update.html

```
{% extends "base.html" %}
{% load tailwind_filters %}
{% block titulo %} agregar Categoria{% endblock titulo %}
{% block contenido %}
    <h1>Actualizar Categoria</h1>
    <form method="post" action="{% url 'update_categoria' categoria.id %}">
        {% csrf_token %}
        {{form|crispy}}
        <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700
        rounded">Guardar</button>
    </form>
{% endblock contenido %}
```

33. hoy solo nos hace falta probar nuesta aplicasion

	**EJERCICIOS DE PRÁCTICA:**

- con todo lo anteriro realizado y apuedo realizar el matenimiento de las tablas
- Tio de pago
- Clientes
- roles